

Prateek Gautam Project List

Data Scientist · Junior Developer Program

Day 1: June 2, 2025

Mentors: Sonal Oberoi and John Abraham

Manager: Sriram Rajagopalan

1) Project: KDP Metrics Dashboard and Enhancement

a) Overview:

- i) Business question / Task: Understand the current state of KDP books across all regions and marketplaces. The team did not have a unified platform to visualize KDP metrics; the dashboard would serve this purpose by not only aggregating information across publishers and vendors but also providing deep dives on KDP ASINs.
- ii) Stakeholders: Product Managers (shaclind@, harstc@)
- iii) Deliverables/Artifacts:
 - (1) [Dashboard](#)
 - (2) [ETL Job for ebook](#)
 - (3) [ETL Job for pbook](#)

b) Methodology/What I did:

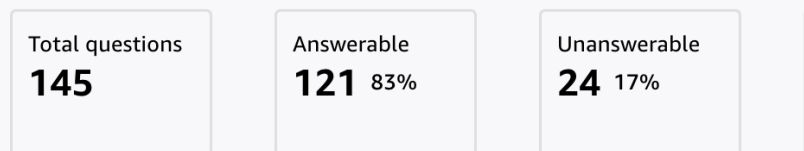
- i) Created master tables for ebooks and pbooks based on the team's source of truth tables.
- ii) Where the data was not readily available, I created entirely new transform/load jobs and datasets on ETL Manager and AWS QuickSight respectively.
- iii) Collaborated with my project mentor and stakeholders to establish milestones and understand what information would be helpful to highlight/visualize.

c) Key findings and Impact:

- i) 12M+ KDP ASINs and 3M+ publisher information across all Amazon region/marketplaces is readily available via the dashboard
- (1) Metadata information: Title Name, Author Name, Price information across different currencies, Language, Genre, Frontlist vs Backlist categorization, Select enrollment, Ratings and Reviews
 - (2) Account information: Publication date and Publisher/Vendor information, Royalty (+bonus) payout information, ALC purchase history, Glance Views and Coverage
 - (3) **Enhancement:** Created a chat topic on AWS Quick for customers to answer dashboard questions without having to manually use filters and navigate through tabs on dashboard.
 - (a) Created relevant data fields, named entities, calculated fields, and custom instructions for the agent (clearly providing context, clarity, and constraints)
 - (b) The KPIs from the last 12 months are provided here (the total questions also include questions asked during testing)

User activity

See everything users asked and how well Q responded for this topic. [Learn more](#)



The above KPIs represent the last 12 months of user activity.

- (c) Upon launch of Stargate, the knowledge base (datasets, fields, suggested questions) for the chat agent was used for seamless migration; no major rework had to be performed for the KDP topic on Stargate.
- ii) Customer Feedback:
- (1) When a teammate inquired about ebook and pbook GMS, obesonal@ mentioned: “You can also use KDP metrics dashboard and chatbot developed by Prateek for calculating ebook and pbook royalties.”
 - (2) Songp@: “[dashboard helped me] to further clarify around KDP author for an analysis I am working on.”

d) Leadership Principle: Invent and Simplify

The dashboard answered every KDP question the PMs had, but at 10 tabs deep, finding the right answer was its own task. The real barrier was not missing data; it was the access pattern. So I built a chat topic over the same datasets and authored the semantic layer behind it: visible fields, named entities, calculated fields, and constraints on what the agent must not guess. It resolves 83% of questions on its own, turning a multi-tab hunt into a single question.

I also designed the semantic layer as a durable artifact rather than platform-specific prompts, which meant it carried to Stargate with no rework. Underneath, the master tables for ebooks and pbooks, plus the new ETL jobs I wrote where source data did not exist yet, are the foundation both the dashboard and the chat topic sit on.

2) Project: KDP to Managed Dashboard and Enhancement

a) Overview:

- i) Business question / Task: Following requests from multiple L8 leaders, KDP and managed teams worked to create queries showing movement of titles from KDP to managed publishers. To track movement over time, there was a request for a Quicksight dashboard with one report for digital and one for print.
- ii) Stakeholders: Product team (iandel@, alterj@)
- iii) Deliverables/Artifacts:
 - (1) [Dashboard](#)
 - (2) [QBR Excerpt](#) - shared by Ian (VP visibility; doc is confidential so just shared the table and relevant callouts)

b) Methodology/What I did:

- i) Used the existing logic based on title_ids to identify KDP and Managed ASINs. Created datasets on AWS Quick based on stakeholder request.
- ii) Since I took over this project from a teammate who transitioned away from our team, the datanet jobs were already in place. However, the jobs were taking a long time to run and would frequently error out (see screenshots in the Impact sub-section below). I optimized the SQL – through a combination of: implementing CTEs for debugging, pushing filters early, and using temp tables instead of complex joins.
 - (1) Screenshot taken from: https://datacentral.a2z.com/datanet/etl-manager/jobs/28176574/runs?time_range=DATE_RANGE&from_date=2025-06-01&to_date=2026-08-31

c) Key findings and Impact:

- i) Enhancements made: Updated the UI (with additional filtering capabilities, created an entirely new “Deep Dive” tab that combines ebook and pbook information) as per stakeholder feedback. Added line graph and tree map visualizations for ASIN and author movements to complement the existing KPI boxes.
- ii) Pre-optimization (July 2025 runs – note the Minutes Run column, ranging from 63 minutes to 108 minutes)

☐	Job Run	Status	Dataset Date	Schedule Hour	Submitted By	Minutes Run	Rows Processed	Dataset
☐	11159766554	Success	2025-07-30	24:00 (00:00)	vemaladd	76	1881	book
☐	11156772392	Success	2025-07-29	24:00 (00:00)	vemaladd	84	1864	book
☐	11158711852	Success	2025-07-29	24:00 (00:00)	vemaladd	77	1880	book
☐	11158790367	Success	2025-07-28	24:00 (00:00)	dwp	88	1880	book
☐	11145235048	Success	2025-07-26	24:00 (00:00)	Scheduled	66	1853	book
☐	11122836698	Success	2025-07-19	24:00 (00:00)	Scheduled	79	1849	book
☐	11100725633	Success	2025-07-12	24:00 (00:00)	Scheduled	108	1905	book
☐	11089822247	Success	2025-07-08	24:00 (00:00)	vemaladd	96	1925	book
☐	11090110017	Success	2025-07-08	24:00 (00:00)	vemaladd	64	1886	book
☐	11087208095	Success	2025-07-07	24:00 (00:00)	vemaladd	63	1867	book
☐	11078776886	Success	2025-07-05	24:00 (00:00)	Scheduled	73	1905	book

iii) Post-optimization (April 2026 runs – note the Minutes Run column, ranging from 24 minutes to 44 minutes):

☐	Job Run	Status	Dataset Date	Schedule Hour	Submitted By	Minutes Run	Rows Processed	Dataset
☐	12235988351	Success	2026-04-25	24:00 (00:00)	Scheduled	24	2059	book
☐	12204183642	Success	2026-04-18	24:00 (00:00)	Scheduled	44	2064	book
☐	12172306786	Success	2026-04-11	24:00 (00:00)	Scheduled	38	2067	book
☐	12140838491	Success	2026-04-04	24:00 (00:00)	Scheduled	38	2070	book

iv) Enhancement in Progress: KDP to Managed ML Model

(1) Objective 1: Predict 1 month in advance if an author is going to transition from KDP to Managed Publishers

(2) Objective 2: Identify the most distinctive attributes of authors who are most likely to transition from KDP to Managed

(3) Status:

- (a) Requirements consolidated to include a two-layer feature spec with 27 features grouped across: 22 book-level performance (velocity, GV, KU, Select enrollment) and 5 author-level context (catalog concentration, gateway).
- (b) Built labels, eligible population, and both feature layers, with several rounds of correctness fixes - most notably catching and removing two temporal-leakage bugs (a risk-set boundary error, and a current-state catalog field that leaked future info)
- (c) Modeling approach on [SageMaker](#): LightGBM trained on Phase 1 with 50:1 downsampling, isotonic calibration fit on Phase 2, single evaluation on Phase 3 holdout.
- (d) The top features are catalog footprint, author GMS, asset maturity, KENPC, and list price - together with a few demand signals (GV velocity, GV percentile, GV, catalog concentration), accounting for ~78% of model gain. Structural/publisher and demand signals dominate over the contagion gateway and velocity features.

v) Customer feedback:

- (1) Ruegerj@: “thank you this is very helpful. it helps my use case as id would like to understand a migration we observed in US. but wondering if there are plans to expand this in the future?”

d) Leadership Principle: Ownership

I took this project over from a teammate who left. The Datanet jobs already existed and the dashboard populated, so on paper it was fine, and every stakeholder ask was cosmetic: more filtering, better visualizations. Nobody asked me to touch the pipeline.

The runs were taking 63 to 108 minutes, which left no slack if an upstream delivery ran late. So I rewrote SQL I hadn't authored: broke it into CTEs to isolate where the time was going, pushed filters as early as possible, and swapped the heavier joins for temp tables. Runtime came down to 24 to 44 minutes, on roughly 10% more rows than before.

I delivered the asks too: additional filters, line graph and tree map views of ASIN and author movement, and a new Deep Dive tab that puts ebook and pbook side by side.

3) Project: Managed to KDP Migration Tracking

a) Overview:

- i) Business question / Task: Like tracking KDP to Managed migrations, the ask was to create a reporting framework for Managed to KDP migrations. Details are in the SIM [here](#).
- ii) Stakeholders: iandel@
- iii) Deliverables/Artifacts:
 - (1) [Dashboard](#)
 - (2) Underlying [ETL job](#)

b) Methodology/What I did:

- i) Created a [baseline master table](#) to track Managed to KDP transitions, ensuring all stakeholder requirements were met.
- ii) Built the dashboard for ongoing reporting with metrics for: number of titles and authors moved, tenure, language breakdown, GMS, COGS and Royalty Breakdown, Qualified Borrows, and Number of Readers/Customers
- iii) Identified that the upstream data sources (indie_unified_metrics) take up to 2 months for the royalty information to update. Thus, the most recent royalties for KDP titles were added as estimates based on the average PPR for US Marketplace in the last 3 months.

c) Key findings and Impact:

- i) Metrics from the dashboard(s) are incorporated into MBR/QBRs [please see the artifact above] and according to Ian is regularly discussed in the financial discussions for the business
- ii) Line chart and tree map visualizations, when used with appropriate date range and metadata filters, provide an aggregated “high-level” view for stakeholders, eliminating the need for repeated querying.

d) Customer feedback:

- i) iandel@: “it's also given numbers to a trend that was previously only anecdotal but very high interest for leadership”

e) Leadership Principle: Bias for Action

indie_unified_metrics settles roughly 45 days behind, so the most recent months of KU royalty are always incomplete. The default

response was to wait for it. That produces a report that is accurate and useless for precisely the period stakeholders ask about. Thus, I decided a defensible estimate now was worth more than an exact number in seven weeks, so I built the fallback into the monthly grain. Because of this implementation, actuals always win; the estimate only fills a NULL. The metric is live on day one and repairs itself as IUM settles, with no backfill job and no intervention from me.

4) Project: Project Iceberg (Phase 1 and Phase 2)

a) Overview:

- i) Business question / Task: As part of the Iceberg program, the business was seeking
 - (1) [Phase 1](#): Latest list for KDP authors that are based on EU27 (per TIMS data) and payments from AMEU company codes (from FinOps data) to perform author outreach.
 - (2) [Phase 2](#): For Publishers with residency in the EU, figure out which marketplace they earn most of their royalty from. This would assist with legal enforcement.

(a) Related SIM for Phase 2:

<https://issues.amazon.com/issues/MINANLYT-4266>

- ii) Stakeholders: shhmira@, shaclind@, briscorn@

iii) Deliverables/Artifacts:

- (1) Requirements doc: <https://quip-amazon.com/OlmQAAcljRVZ/KDP-Iceberg-Author-Data>
- (2) Phase 1 data export:
s3://icebergdatasetspgtm/consolidatedData/consolidated_data_iceberg_final.csv
- (3) Phase 2 data export: s3://iceberg-q1-2026/Account_and_COR_details.xlsx
- (4) Final [Workbench](#)

b) Methodology

- i) Understood the data requirements and communicated the limitations and data quality issues regarding:
 - (1) Vendor code, un-encrypted Customer Id, un-encrypted account ID, preferred language, ETA flag, Account status
 - (2) Permanent country code and US interview Expiry date (From TIMS) and Payments from AMEU (from FinOps) for Trailing Twelve Months
 - For instance, ETA flag is NULL for legacy accounts that were grandfathered in rather than explicitly accepting. Treating NULL as "not accepted" would have queued long-standing compliant authors for payment block, and nobody would have caught it until an author complained about being blocked for

a form they were never asked to sign. Flagging it forced a three-state read: accepted, not accepted, unknown, with unknown routed to outreach instead of enforcement.

- ii) Collaborated with FinOps and TIMS to get the required data and ensure that the project timelines were not affected

c) Key findings and Impact:

- i) The data exports fed the rollout: beta-I to 1K authors, beta-II to 25K, GA without enforcement, then GA with enforcement.

d) Leadership Principle: Earn Trust

Iceberg depended on data my team didn't own. Residency sat with TIMS, payments sat with FinOps, and I had no authority over either, so credibility was the only thing that actually moved the work. I set up the working sessions myself and put the requirements in writing, so both partner teams and my stakeholders could hold me to one document rather than to three versions of a verbal ask. When the picture changed, they heard it from me first.

The harder conversation was about my own deliverable. The sources carried gaps and reliability limits, and I could have handed over a clean-looking file and let those surface after someone had already built on it. I raised them while stakeholders were still deciding how to use the data, because a limitation disclosed early is something they can design around, while the same limitation disclosed late is just an explanation for a decision that already went wrong.

5) Project: Royalty Backfill Request

a) Overview:

- i) Business question / Task: As part of legacy deprecation, stakeholders wanted to spot check the fidelity of accounts that had metadata backfilled. However, there's a list of 500K vendor codes in 4 separate files (making it hard to self-serve) that we need royalty information for.
- ii) Stakeholders: lael@, shaclind@
- iii) Deliverables/Artifacts:
 - (1) SIM: <https://issues.amazon.com/issues/MINANLYT-4236>
 - (2) Final [Workbench](#)

b) Methodology

- i) Extracted account_ids from 4 different JSON files provided by Mesa team using a Python script and consolidated them into a single txt file for efficient analysis
- ii) Used the COPY command (lines 15-20 in the code Workbench above) so that we don't have to use a lengthy WHERE clause or an INSERT INTO statement.
- iii) Calculated ebook and pbook royalties for the consolidated account_ids

c) Key findings and Impact:

- i) Confirmed 1:1 mapping for *accountids and vendorcodes*.
- ii) Identified the top 10 vendors as requested; confirmed the backfill was accurate.
- iii) Provided a solution to self-serve large queries.

d) Customer feedback:

- i) Shaclind@: "QQ to Sriram and Prateek - Is there a way for us to self serve for large queries like this going forward so we don't have to keep bugging you?" -> provided the details on the COPY command use via S3 bucket and the relevant Python script for consolidation as a followup
- ii) Lael@: "Thanks Prateek!! Appreciate your fast turnaround for this."

e) Leadership Principle: Customer Obsession

Legacy deprecation had a hard deadline, and the Mesa team's royalty data was scattered across four separate JSON files with overlapping keys. If a stakeholder had to wait for someone to manually reconcile

those files, the spot check would have slipped, and downstream publishers would have had no confirmation that their royalties were accurate. I started from that risk: publishers deserve timely proof that their payouts are correct, so I built a pipeline that deduped the four files into a single joinable table, caught the overlapping keys that would have silently inflated totals, and delivered the top-10 vendor validation same week. But the real customer-obsessed move was what came next. When Linda asked whether the team could self-serve large queries like this in the future, I already had the answer ready, because I had designed the pipeline to be reusable rather than disposable. Removing that recurring dependency meant the next request would not have to wait on me or anyone else.

6) Project: KDP Select Bonus Revamp

a) Overview:

i) Business question / Task:

- (1) Examine whether the current bonus allocation methodology appropriately rewards publishers based on customer engagement patterns across different scenarios.
- (2) Evaluate the effectiveness of the existing bonus distribution approach through hypothetical scenario modeling.

ii) Stakeholders: shaclind@

iii) Leadership: kronbach@, anndav@, zimpel@

iv) Deliverables/Artifacts:

- (1) Doc: [KDP Select All Stars Bonus Revamp - Dec16 Review.docx](#)
- (2) Vendor Level [Analysis](#)
- (3) ASIN Level [Analysis](#)

b) Methodology

- i) Created a cohort of interest: All KDP Select ASINs with at least a page read in the last 12 months
- ii) Calculated completion rate (This value does not count customer re-reads more than once) and split the cohort on pages read buckets with breakdown for ASIN counts, average completion rates, and number of readers
- iii) I compared multiple allocation scenarios to assess the impact of different attribution models on vendor compensation, with particular attention to how each approach rewards publisher engagement with customers.

c) Key findings and Impact:

i) Quantified the current state:

- (1) Average completion rate of a KDPS title is 52.06%
- (2) Out of all the books in KDP catalog, only 37 titles have more than 90M+ pages read
- (3) 1,977 vendors earned a bonus in the month this analysis was performed

ii) Performed hypothetical scenario modeling to arrive at these conclusions:

Hypothetical Reward Scenario	Number of vendors that will qualify	Average change in ASIN Bonus (for qualifying ASINs)
First Borrow (Converted Customer From Free Trial to Paid Sub)	2,466	+\$88.91
Last Borrow (Converted Customer From Free Trial to Paid Sub)	2,284	+\$72.42
First Borrow by Paid Subscriber	2,608	+\$89.51
ASINs that engage churn risk customers	2,424	+\$84.23
First Borrow Ever by All Customers	2,836	+\$104.13

d) Leadership Principle: Think Big

The KDP Select bonus pays on pages read, which sounds fair until you look at the distribution. Approximately 5k titles in the entire catalog earn a bonus at all, and just 37 clear 90M pages read. A volume basis concentrates a large fund on a handful of outliers, which is defensible but not obviously what KU most needs to reward.

So instead of tuning the existing formula, we asked what the bonus should be buying and modeled five structurally different allocation bases, each rewarding a customer behavior rather than volume. My contribution wasn't picking the winner. It was making the choice arithmetic instead of intuition, so Books leadership could debate which customer behavior the bonus is meant to reinforce rather than whether the current split feels right. **That decision is open/pending as of this writing.**

7) Project: e-Reader Deprecation

- a) Overview: When Amazon retired 9 legacy Kindle e-readers on 22 May 2026, those devices stopped emitting reading signals.
 - i) Business question: How much royalty was tied to reading on the retired devices?
 - ii) Stakeholders: shaclind@, johnabrm@ (technical review)
 - iii) Leadership: kronbach@, anndav@, zimpel@
 - iv) Deliverables/Artifacts:
 - (1) Doc: [Ereader_Summary_August28.docx](#)
 - (2) Marketplace level royalty payment summary
 - (3) Customer × asin × marketplace detail
 - (4) Data dictionary: [ereader_task1_data_dictionary.md](#)
- b) Methodology
 - i) Measured unread payable KENP on the cohort's active borrows at deprecation day (2026-05-22), then again three months later (2026-08-27). Whatever stayed unread is royalty that had not yet been paid. Priced it at verified per-marketplace local rates, and split each borrow by whether reading continued elsewhere - separating pages that stopped from pages that simply moved to another device.
 - ii) What I did:
 - (1) Owned the analysis end-to-end: cohort construction, query design, data quality checks, and the deliverables.
 - (2) Reworked the pipeline after review feedback - moved marketplace to the transaction grain (cut cross-marketplace customer overlap to 1 of 73,656), replaced a multiplying equi-join with a dup-proof semi-join, and added a hard grain assertion as a gate before expensive steps of andes table scan.
 - (3) Introduced per-marketplace local-currency PPRs with FX rather than a flat WW rate, materially sharpening per-marketplace figures.
- c) Key findings and Impact:
 - i) Only 6% of the deprecation-day unread KENP was billed in the following 3 months - reading did not visibly migrate for the vast majority.
 - ii) KDP royalties owed are ~\$263K

- iii) Confirmed that the KU reader retains the book in their library even if the book eventually gets unpublished. These titles are also accessible on any device that is logged into the same KU account.

d) Leadership Principle: Insist on Highest Standards

On marketplace: v1 read it off the cohort file, which lists the same customer_id under several country codes and quietly inflated activity. Taking John's feedback, I cut the cohort down to distinct customer_ids and sourced marketplace from the transaction itself, carrying marketplace_id through the borrow-to-download join. The check that it worked: cross-marketplace customer overlap fell to exactly one customer out of 73,656.

On rate: v1 priced every marketplace at the worldwide PPR of 0.00427 USD. Against the total that looks harmless, \$267,998 versus the corrected \$262,695, about 2%, because the flat rate happened to sit near the volume-weighted average. Per marketplace it was wrong everywhere: US is 0.00484, JP is 0.36371 JPY, roughly 0.00231. I replaced it with local PPRs from row 96 of the July fund doc, verified against the KDPS Finance Model, applying FX at customer-ASIN-marketplace grain so a roll-up crossing currencies still sums correctly.

And rather than one headline number, I gave the PM four tiers with the reasoning for each.

8) Project: [AI initiative] meshClaw agent (hosted on Slack) for KDP-KU

a) Overview:

i) Business Context / Task:

Two challenges: First, fund-doc sections were slow and person-dependent: each took up to a day to produce, between running the numbers, checking them, and formatting the output - and if the owner was out of office, the fund doc waited. Second, the team's analytical context was locked in a repo, so the only people who could reach it were the people already working inside it.

The task was to make the knowledge base answerable from Slack, and make the recurring fund-doc work reproducible by something other than one person's memory.

ii) Stakeholders: shaclind@

iii) Deliverables/Artifacts:

(1) Agent: [meshClaw-KDP-KU on Slack](#)

(2) Business facing [document](#)

(3) Custom created [spreadsheet-mcp](#)

(4) [Presentation deck](#) for AI Learning Series on September 2, 2026 with the demo screenshots attached

b) Methodology/Details:

- i) Read-only by construction. SELECT-only on the query path, GetObject on storage, no write path anywhere - enforced in the credential and the tool, not in a prompt.
- ii) Nothing upstream changes. The finance model stays a spreadsheet, the analyses stay in the repo, people stay in Slack.
- iii) Skills carry their own verification. Each holds the prior month's values as a baseline, so a rebuild that can't reproduce them is known-wrong before anyone reads it. Where two systems measure the same quantity, the skill reconciles them rather than trusting either.
- iv) Prior conclusions before new queries. The agent searches existing analyses first, which lets it correct a question's premise instead of only answering it.
- v) An acknowledged gap beats a plausible number. No estimating, no interpolating, no carrying a month forward; every figure names its source, and a failed fetch hands over the query and the dashboard link. A

confidently wrong number in a finance doc costs more than a missing one.

c) Key findings and Impact:

- i) Fund-doc sections now assemble from a single Slack message. For instance, KDPS Health returns in about 90 seconds against up to a day by hand - three QuickSight visuals, 1,170 rows, 13 marketplaces, reshaped into the fund-doc layout with month-over-month and year-over-year computed.
- ii) Removed a single-person dependency from the fund doc. The section methods are encoded, so Linda is no longer blocked on engineers being available (for instance, Praveen was out on PTO for 3 weeks). This matters more than the time saved.

d) Leadership Principle: Insist on the Highest Standards / Invent and Simplify

The invention was not the agent itself; it was encoding the fund-doc methods so they could run without the person who wrote them. By having the agent search existing analyses before running new queries, it can correct a question's premise rather than just answering it, and the entire KDPS Health section now assembles from a single Slack message instead of a day of manual work. That simplified a workflow that had been blocked every time its owner was unavailable.

The harder part was making the answers safe enough to put in a document shared across teams. I built validated baselines that act as regression tests on the data, a cross-source reconciliation between two independent systems, a ratio guard that catches a wrong-column read in one division, and explicit warnings for the three columns that resemble the royalty figure but are not, one of which is off by $2.5\times$ and still a plausible dollar amount. The guardrail worth writing is the one that catches a plausible error, not an obvious one.

9) Project Template:

a) Overview:

i) Business question / Task:

ii) Stakeholders:

iii) Deliverables:

b) Methodology

i) What I did:

c) Key findings and Impact:

d) LPs learnt:

10) Projects where I contributed to some part(s) of a larger task

a) **SalesRank Investigation:**

i) Overview: Several top KDP author complaints were received on title visibility issues, Sales Rank concerns, and sales performance changes.

ii) Artifacts:

(1) [Investigation Document](#)

(2) [Internal Documentation](#)

(3) [Workbench \[for monthly GV Conversion\]](#)

(4) [Workbench \[for aggregated author monthly earnings\]](#)

iii) What I did:

(1) Verified the Glance View Coverage and tiered earnings of KDP authors

(2) Quantified and visualized the ALC vs KU conversion for the publishers in investigation

b) **Prime Reading ETL job failure:**

i) Overview: Resolved critical 0-rows issue affecting 10 marketplaces despite unfamiliarity with Prime Reading business and codebase.

ii) Artifacts:

(1) [Prime Reading Transform Job for US](#)

(2) [Prime Reading Transform Job for JP](#)

(3) <Plus 8 other marketplaces>

iii) Leadership Principle: Deliver Results

Praveen flagged this as high priority: 10 marketplaces returning zero rows and no one immediately familiar with the root cause. I had never seen the 400-line Prime Reading codebase or its underlying tables, so I started from the query logic and tested each step until I isolated the failing join. Root cause identified within 8 hours, fix shipped by the next business day, and all 10 marketplace pipelines restored before the outage could cascade into downstream reporting.

c) List Price Change for KDP Books

- i) [Overview](#): Starting July 7, 2026, the maximum list price for the 70% royalty option price band changed from \$9.99 to \$12.99 on Amazon.com, with equivalent changes across all KDP marketplaces.
- ii) Stakeholders: harstc@, shhelmun@
- iii) Artifacts:
 - (1) [Workbench](#) to identify e-book repricing
 - (2) [ETL Profile](#) for ongoing reporting as requested by stakeholders
- iv) What I did:

I navigated through unclear requirements to provide the core data requests - ASINs that had a change in royalty plan post 7/7 launch, and ASINs which have re-priced since 7/7.

 - (1) Built a per-ASIN price-change log for US KDP ebooks covering the 7 July change to the 70% royalty band with one row per material change plus a baseline row, emitted only for ASINs that actually moved.
 - (2) Productionized it as an ETL Manager job (profile 44062518) writing to kdp_ddl.kdp_ebook_repriced, so stakeholders query a table on a daily refresh cadence instead of asking me for SQL
 - (3) I actually discarded my first design with feedback from my mentor and manager. A two-point before/after comparison couldn't say when a price changed, and silently missed ASINs that moved twice and netted to zero. Hence, I rebuilt it as an event log using LAG over daily snapshots.

d) Praveen Coverage tasks during PTO:

i) Rosetta Translation Reports Changes Implementation

- (1) Artifacts:
 - (a) SIM ticket [here](#)

- (b) [V2 Transform Job](#) that is currently onboarded
- (2) Stakeholders: jecmpb@
- (3) What I did:
 - (a) Took the Rosetta translation-log schema change from proposal to production. The pipeline reads a daily JSON dropped to S3, lands it in Redshift, and feeds the Rosetta beta dashboard and Page 0.
 - (b) Rewrote the SDL against the v2 JSON supplied by KDP Tech (jecmpb@). v2 isn't additive, it re-nests: v1's sibling summary blocks collapse into translation_summary.translation.* stages plus a new ne_research branch, and per-stage cost_estimate and model_id replace token-only aggr_inference_stats. That's why the downstream cost analysis was exposed - cost stops being derived from token counts and becomes a supplied figure per stage.
 - (c) Sequenced the cutover around a mid-day migration on 4 August. Descheduled the v1 job first, ran a transition job across the changeover so that day's file - part v1, part v2 - wasn't half-lost, then descheduled the transition job and brought the v2 job to production once migration completed.
 - (d) Kept deprecated columns in the Redshift table hard-coded null instead of dropping them, so Page 0 and the cost analysis didn't break on missing fields. Likewise, made the load type-safe by explicit casting to match the target DDL precision.
 - (e) Verified outputs against the v2 JSON with the KDP Tech point of contact before closing the change.

ii) Tandem Royalty Fix on Dashboard:

- (1) Artifacts:
 - (a) [Dashboard](#)
- (2) Stakeholders: lynmargo@
- (3) What I did:
 - (a) Identified three independent defects which caused the distortion for royalty metrics on QuickSight:

- (i) The join didn't match on date. Rows were being paired across mismatched periods, so the royalty total was assembled from the wrong slices.
 - (ii) A run ID wasn't excluded, which double-counted royalties. That exclusion couldn't be derived from the pipeline or its documentation. I picked the run ID up from Finance while working on an unrelated project and recognized it applied here.
 - (iii) The refresh was set to weekly against a monthly grain. Indie unified metrics are monthly, so a weekly schedule was re-slicing a monthly figure.
- (b) Corrected all three and shipped the fix, so the dashboard reports author royalty accurately.

Other Notes

Relevant Trainings & Certifications via Amazon Learn

1. Python Refresher: Recall & Code
2. Data Analysis eLearning Path - Advanced - GA - Data Automation & Analysis using Python & Pandas
3. In Progress: AWS Machine Learning Associate Certification (Estimated completion: December 2026)